MiTDM: Eliminating False Conflicts in Scalable MVCC in Disaggregated Memory

Anonymous Author(s)

ABSTRACT

Disaggregated memory architecture has gained wide adoption in cloud and high-performance systems [18, 29, 41] due to its decoupled resource model, elasticity, and low-latency access. In such architectures, transaction mechanisms must ensure atomic and consistent access to remote memory. Prior designs use array-based version structures to improve locality, but their fixed capacity introduces false conflicts, especially for long transactions that require accessing historical versions, limiting concurrency and throughput.

We present **MiTDM**, a transaction system tailored for disaggregated memory environments. MiTDM introduces a *hierarchical block version chain* that combines the benefits of array and chained structures, supporting dynamic version expansion while maintaining low-latency access. A *lightweight Block Index Table* accelerates remote reads, and an *asynchronous index update* mechanism synchronizes indexes across nodes with minimal overhead. To manage version growth, MiTDM employs a *hybrid garbage collection* strategy aware of transaction states, enabling safe reclamation and reuse of version slots, particularly in hot spots. A *one-sided optimistic transaction protocol* ensures consistency and atomicity under high concurrency.

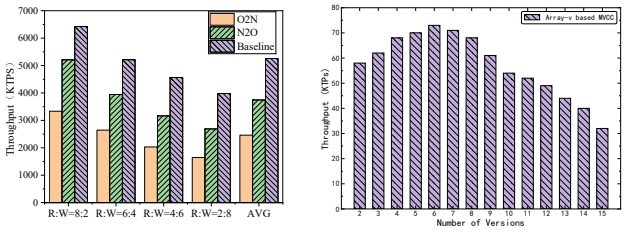
Experimental results on a disaggregated memory cluster show that MiTDM outperforms state-of-the-art systems. Under the TPC-C workload with 128 clients, it achieves up to **80.1% higher throughput** compared to FaRMV2 and **42.7% higher** than Motor, while significantly reducing P50 and P99 latencies.

ACM Reference Format:

Anonymous Author(s). 2025. MiTDM: Eliminating False Conflicts in Scalable MVCC in Disaggregated Memory. In *Proceedings of ACM Conference Name (Conference'25)*. ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/xxxxxxx.xxxxxxx

1 INTRODUCTION

Disaggregated memory (DM) decouples compute and memory to improve elasticity and utilization in modern data centers [8, 15, 18, 29, 41], enabling independent scaling of memory pools while reducing failure domains [2, 40]. Modern databases widely adopt Multi-Version Concurrency Control (MVCC) [30, 34] for snapshot isolation. However, traditional MVCC designs, optimized for local memory, suffer severe performance degradation in DM because remote access via RDMA (Remote Direct Memory Access) or CXL (Compute Express Link) turns inexpensive local operations into costly network round-trips.



(a) Throughput of pointer-based version chains under varying write intensities.

(b) Throughput of static array-based MVCC as version capacity varies.

Figure 1: Version management trade-offs in DM. (a) Chain-based throughput under varying write intensity. (b) Array-based throughput against version capacity.

Many MVCC systems organize historical versions as pointer-linked chains, which perform poorly in DM due to iterative RDMA operations—each pointer dereference triggering a separate remote read. In our RDMA testbed, a single-step version traversal takes 3.05 μs, whereas traversing a 20-version chain requires 74.03 μs—a 24.2× increase. As Figure 1a shows, this translates to a 60% throughput degradation under high-contention YCSB workloads.

Recent designs organize versions in contiguous arrays [31, 38], enabling batched retrieval via a single RDMA read. However, these layouts suffer from a rigid *static-capacity trade-off*: **On the one hand**, small arrays keep each read compact but evict still-visible versions under long snapshots, forcing *capacity-induced false conflicts* in scenarios like HTAP workloads [6, 11], consistent scans, and skewed hot-key access; **on the other hand**, larger arrays reduce such aborts, but every read must load the whole enlarged version array back from remote memory, wasting bandwidth and lowering throughput. In our evaluation, 3-version Motor incurs 30% false conflicts under only 6% long-running queries (Figure 2). Thus, chains suffer prohibitive latency, while static arrays must choose between high conflicts with few version slots and low throughput from full-array loads with many version slots.

To address this, we propose **MiTDM**, a DM-native system built upon a **Hierarchical Block-based Version Chain (HBVC)**. HBVC groups versions into fixed-size blocks linked across blocks rather than individual versions: within a block, versions are batched in one read; across blocks, the chain grows elastically. A lightweight **Block Index Table** caches overflow-block addresses for records that have expanded beyond the inline block, allowing readers to issue a small number of batched RDMA reads. Realizing this in DM introduces challenges in minimizing *data transfer overhead*, ensuring *metadata synchronization*, and managing *passive garbage collection*—each addressed in Section 3. In this paper, we make the following contributions:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'25, Month Year, City, Country

© 2025 Association for Computing Machinery.

ACM ISBN 978-x-xxxx-xxxx-x/xx/xx…$15.00
https://doi.org/10.1145/xxxxxxx.xxxxxxx

- We identify the fundamental trade-off in DM version management: the latency of pointer chasing versus the false conflicts of static arrays. We introduce HBVC to reconcile these conflicting requirements.
- We present MiTDM, incorporating batched retrieval, asynchronous updates, and transaction-aware GC to resolve these challenges.
- Evaluation results demonstrate that MiTDM outperforms state-of-the-art array-based systems by up to 3.2× in throughput under high contention, while eliminating capacity-induced false conflicts in our evaluated workloads.

2 BACKGROUND AND MOTIVATION

2.1 Disaggregated Memory Characteristics

Disaggregated memory (DM) decouples compute and memory into scalable pools connected via RDMA or CXL [7, 8]. While enhancing elasticity, DM introduces high remote access latency dominated by network round-trips (RTTs). A critical constraint is *compute asymmetry*: passive memory pools lack the processing power for complex logic like garbage collection or pointer chasing. Consequently, transactions must rely on compute-side orchestration. We identify three critical design requirements:

- **Minimize dependent RTTs:** Version traversal should avoid serial network round-trips caused by address dependencies (e.g., pointer chasing).
- **Avoid memory-side CPU reliance:** The design must operate on passive memory pools, precluding background GC, compaction, or complex coordination logic on memory nodes.
- **Control bytes per lookup:** The system should avoid rigid read amplification (e.g., globally large fixed arrays) and keep transferred bytes proportional to the versions that are actually needed.

2.2 Limitations of Existing MVCC Designs in DM

RDMA-based systems reveal fundamental versioning trade-offs. **Single-version systems** (Xenic [27], FaRM [5], Scythe [21], FORD [39], DrTM [3]) simplify persistence but sacrifice concurrency under contention—a key DM workload characteristic [10, 36]. In contrast, existing MVCC layouts face a fundamental dilemma in DM: they must choose between low latency and elastic capacity. No existing design satisfies both while providing efficient garbage collection.

The Latency Trap of Chained Versions. Traditional designs (e.g., FaRM-V2 [28], NAM-DB [37]) link versions via pointers. While offering unbounded capacity, this incurs *dependent RDMA reads* in DM: the next version's address is unknown until the current read completes. This serialization causes latency to scale linearly with chain length, violating the *Minimize dependent RTTs* requirement. As shown in Figure 1a, traversing 20 versions incurs a 60% throughput degradation.

The False Conflicts of Fixed Arrays. Systems like Motor [38] use fixed-size arrays for single-RTT retrieval but suffer from *false conflicts*. We define a false conflict as an abort caused by the eviction of a version still visible to an active snapshot (i.e., its timestamp is newer than the global GC watermark). This forces valid read-only transactions to abort due to storage limits rather than data contention. Moreover, fixed arrays force a trade-off between false conflicts and read amplification, leading to rigid read amplification and poor bandwidth efficiency.

Garbage Collection Overhead. Both layouts struggle with reclamation due to the *Avoid memory-side CPU reliance* constraint. Lacking memory-side logic, GC requires expensive cluster-wide coordination to determine version visibility. This interferes with foreground transactions, creating a dilemma: existing designs cannot simultaneously achieve low latency, elastic capacity, and lightweight GC.

2.3 Motivation: The Case for Hybrid Layout

As discussed above, designing version storage for disaggregated memory (DM) involves a fundamental trade-off between access latency and storage elasticity.

- **Chain-based layouts** offer infinite capacity but suffer from high latency due to pointer chasing (multiple RTTs).
- **Array-based layouts** (e.g., Motor [38]) achieve optimal one-RTT access but are constrained by fixed capacities.

To quantify the limitations of array-based approaches, we analyze Motor under write-intensive workloads. As shown in Figure 2, Motor's fixed-size arrays create severe *false conflicts*. When the array fills up, valid old versions are prematurely evicted, forcing long-running transactions to abort solely due to physical storage limitations rather than logical conflicts. This makes fixed-array designs brittle under skewed writes and long-running snapshots.

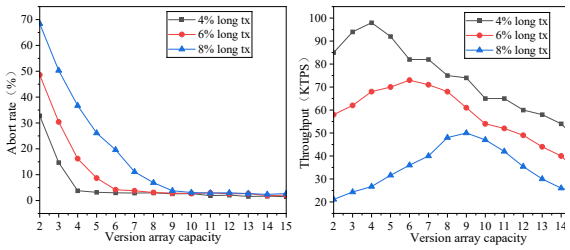


Figure 2: Motor analysis: fixed capacity causes high abort rates (false conflicts) under intensive writes. "x% Long Tx" denotes transactions with tens of operations, analogous to TPC-C Stock-Level (~ 4% of the standard mix).

To resolve this dilemma, we propose **Hierarchical Block-based Version Chain (HBVC)**—a hybrid design that organizes versions into coarse-grained blocks. *Macroscopically*, HBVC behaves like a chain, providing elastic capacity through dynamic block allocation. *Microscopically*, each block acts as a contiguous array, enabling batched RDMA retrieval. This transforms fine-grained per-version pointer chasing into coarse-grained per-block traversal, amortizing network latency while preserving storage elasticity. However, realizing it in a shared-nothing disaggregated architecture requires addressing three system-level challenges.

2.4 Challenges in Disaggregated Implementation

Implementing HBVC efficiently in DM requires bridging the gap between its logical structure and the physical constraints of disaggregated hardware:

- **Data Transfer Efficiency:** Managing the RTT–bytes trade-off: block-based fetching must control read amplification. We address this via a *two-level layout (inline + overflow)* and a compute-side *Block Index Table* that caches overflow-block addresses for multi-block records.
- **Metadata Synchronization:** Synchronizing dynamic **Block Index Tables** across compute nodes without high latency. We use *asynchronous notification queues (NQ)* with a *repair-on-read* mechanism.
- **Passive GC Complexity:** Reclaiming discrete overflow blocks on passive memory without interfering with foreground tasks. We employ a *Global-OAT* watermark and a non-intrusive *reuse* strategy.

3 DESIGN

3.1 Overall Architecture

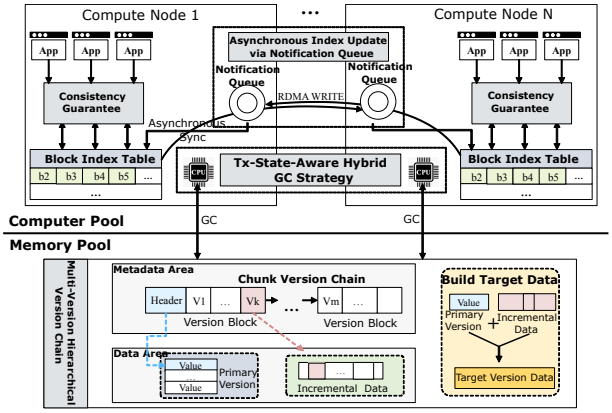


Figure 3: Overview of MiTDM architecture. Compute nodes orchestrate metadata via BIT and ANQ while accessing disaggregated memory pools entirely via one-sided RDMA, utilizing the Hierarchical Block-based Version Chain (HBVC) to balance low-latency 1-RTT reads with unbounded elasticity.

MiTDM is designed as a high-performance MVCC engine specifically for disaggregated memory (DM) environments. It treats remote memory nodes as passive storage pools, accessed exclusively via one-sided RDMA primitives [13, 25] to bypass remote CPU bottlenecks. To resolve the conflict between access latency and storage elasticity identified in Section 2.2, MiTDM introduces a modular architecture centered on the **Hierarchical Block-based Version Chain (HBVC)**.

At its core, MiTDM resolves a fundamental dilemma in DM databases: chain-based MVCC designs (e.g., FaRMV2, NAM-DB) offer elastic history but suffer from dependent RDMA RTTs due to remote pointer chasing, whereas array-based MVCC layouts (e.g., Motor) achieve one-RTT reads but cause false conflicts—capacity-driven aborts that overwrite still-visible versions under long snapshots or hot keys. MiTDM's HBVC addresses this by being array-like within a block (enabling batched one-sided RDMA reads) and chain-like across blocks (allowing dynamic growth). This organization eliminates false conflicts without incurring per-version pointer chasing. To maintain efficiency and correctness at scale, MiTDM pairs HBVC with a lightweight Block Index Table (BIT) updated via asynchronous propagation, and a Global-OAT-guided hybrid garbage collection scheme for safe reclaim and reuse on passive memory nodes.

As illustrated in Figure 3, MiTDM consists of three key architectural layers:

- **Data Layout (HBVC):** Stored in the memory pool, this structure combines a fast-access **Inline Block** with a dynamic chain of **Overflow Blocks**. This hybrid layout ensures hot records are retrieved in one RTT, while older versions remain accessible without fragmentation.
- **Metadata Management:** Compute nodes maintain a lightweight **Block Index Table (BIT)** to locate remote blocks. MiTDM decouples index consistency from transaction correctness using an **Asynchronous Notification Queue (ANQ)**. Memory allocation is decentralized via local bitmaps.
- **Lifecycle Management:** Offloaded to compute nodes, a **State-Aware Hybrid GC** uses global watermarks to reclaim obsolete versions and compact chains, avoiding blocking foreground transactions.

Transaction Workflow. The system operates as follows: (1) Transactions compute the tuple header address via local hash index and probe the BIT. (2) A **BIT hit** batched-reads the indexed Overflow and Inline Blocks. A **BIT miss** fetches the Inline Block and traverses Overflow Blocks step-by-step. (3) Writers acquire per-record locks via RDMA CAS, allocate slots from pre-partitioned regions locally, and asynchronously notify peers. (4) A background GC thread is triggered when the block index reaches a threshold and compacts invisible chains.

This design ensures the transaction protocol (Section 3.6) performs remote data access mostly in batched, one-sided RDMA operations, achieving the goals in Section 2.2.

3.2 HBVC Layout

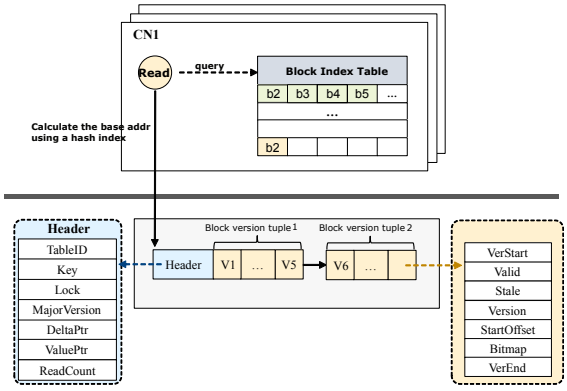


Figure 4: The Hierarchical Block-based Version Chain (HBVC).

The **HBVC** bridges the gap between pointer-chasing lists and contiguous arrays in disaggregated storage. Unlike traditional locality-aware designs targeting CPU cache lines, HBVC is fundamentally

optimized to eliminate dependent RDMA RTTs in passive memory pools where remote CPU assistance (like traversal or active compaction) is unavailable. It organizes version storage into a coarse-grained, two-level hierarchy.

An **Inline Block** located immediately after the tuple header resides in a globally pre-allocated, contiguous region immune to GC. It serves as the permanent anchor of the version chain. To support unbounded histories, it links to larger **Overflow Blocks** (Figure 4). This strategy transforms fine-grained pointer chasing into coarse-grained block scanning. Because RDMA reads of 64B and 256B incur almost identical latency, this coarse-grained layout achieves a free speedup compared to pointer-chasing lists.

3.3 Read Correctness and Payload Layout

For read correctness, each block records occupied slots in a bitmap, determining the active dataset. We separate visibility semantics from atomic integrity:

MVCC Visibility vs. Atomic Read Check. MVCC visibility is determined solely by the version's commit timestamp: a reader with start timestamp T_{start} selects the newest version whose commit timestamp is no larger than T_{start}. The `VerStart` and `VerEnd` fields are not visibility bounds; rather, they are monotonically increasing integrity counters (checksum/seqlock counters) that protect lock-free one-sided RDMA reads from concurrent writes. A reader accepts a slot only when the two counters match and the payload pointer/checksum validation succeeds. If a writer is updating the slot, or if the reader observes a torn mix of old and new metadata, the coordinator treats the read as invalid, aborts the transaction, and releases any acquired references. Thus, timestamps dictate logical visibility, while the start/end counters provide a seqlock-style integrity check for non-blocking 1-sided reads.

Delta-aware Storage. To optimize network bandwidth, the memory pool physically separates compact version metadata, materialized main records, and reverse-delta payloads. The head version is fully materialized in the main-data region, while historical versions store only deltas in the delta region. Metadata slots hold pointers to these payloads, so readers first fetch compact metadata blocks and retrieve main records or deltas only when reconstruction is needed, reducing remote bandwidth for long histories.

3.4 Fast Lookup via BIT and ANQ

The index serves strictly as a performance optimization (hint) rather than a source of truth for correctness, maintaining high fault tolerance. Internally, each locally cached BIT entry maintains the record ID and a dynamically sized array of Overflow Block addresses, so it is not constrained by the compact message format used for inter-CN notification.

Hybrid Reading Workflow (BIT). To safely resolve stale BIT entries, readers proceed via a rigorous multi-step fallback protocol (Figure 5): (1) **Probe & Batch Read:** The reader queries the BIT and issues batched RDMA `READ`s for the Inline Block and any cached Overflow Block addresses. (2) **Validate Structure:** Fetched blocks are validated against the linked structure of the version chain. Starting from the first overflow-block address recorded in the tuple header, the reader follows `next_block` pointers and checks whether each reachable successor is present in the fetched set until reaching

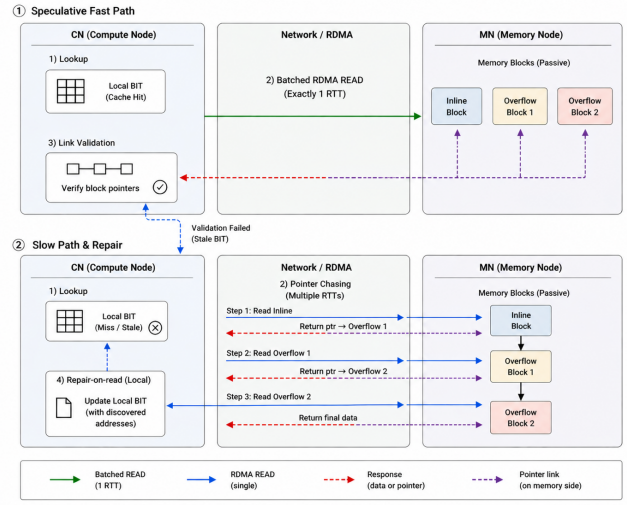


Figure 5: Hybrid Reading Workflow in MiTDM.

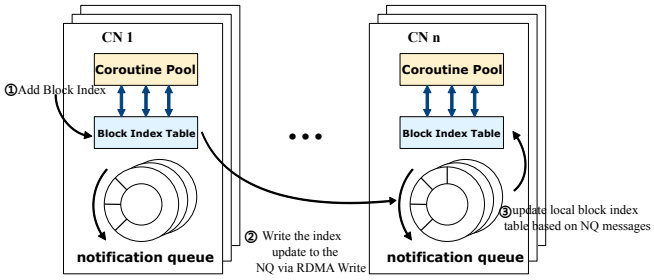


Figure 6: Index synchronization mechanism via Asynchronous Notification Queues: (1) Local update of BIT on modified node, (2) RDMA CAS reserves a remote NQ slot and RDMA WRITE fills the update record without blocking tx commit, (3) Passive receiver application polling with repair-on-read for stale entries.

a `NULL` tail pointer. If the reachable chain is incomplete, the fast-path result is treated as stale or partial. (3) **Fallback (Slow Path):** If the chain is incomplete, the reader resumes standard pointer-based traversal via subsequent RDMA reads, starting from the break point and fetching the missing suffix block by block. (4) **Repair BIT:** After the complete valid chain is observed, the reader executes a "repair-on-read" to update the local BIT.

Index Synchronization (ANQ). (1) When a compute node (CN) modifies HBVC, it updates its local BIT immediately. (2) It broadcasts the change via asynchronous NQs by first using RDMA `CAS` to reserve space at each remote queue tail globally, then issuing RDMA `WRITE` to fill the reserved update record. Each asynchronous notification message is compact (24 bytes), containing only the TableID, Key, and the single newly allocated remote block address. This lock-free concurrent queue handles multi-writer conflicts effectively. (3) Peers passively poll queues and apply updates as speculative hints. Missing, stale, or delayed notifications do not affect correctness,

because readers validate the linked block chain and fall back to the slow path when necessary.

BIT Sizing. MiTDM adopts sparse indexing: only records that have expanded to two or more blocks are inserted into the BIT. Records without such entries directly use the slow path from the inline block. This keeps the local index lightweight while preserving correctness, because the BIT remains an optional acceleration structure rather than a required metadata source.

3.5 Allocation and GC Safety

Memory Allocation. Remote memory is strictly partitioned across CNs. Each CN allocates new Overflow Blocks only from its assigned remote partition by scanning a local bit array, avoiding remote coordination. Crucially to hardware constraints, for each key, newly allocated Overflow Blocks must be constrained to the same physical memory node as the existing chain. This is because RDMA doorbell batching cannot merge requests targeting different memory nodes (a hardware limitation for 1-RTT batch reading).

Garbage Collection Safety. A CN's **Local-OAT** explicitly tracks the minimum start timestamp among all its currently active transactions. If a CN has no active transactions, the Local-OAT simply advances to the latest committed timestamp visible on that node, ensuring the watermark increases monotonically without freezing the system during idle periods. CNs periodically exchange Local-OATs to establish the **Global-OAT**, computed globally as $\min(\text{Local-OAT}_i)$. By definition, versions older than the Global-OAT are logically invisible across the cluster. Physical reclamation is coordinated at the record header: the background GC proceeds only when the header lock is free and the header-level `read_cnt` is zero. This read-reference counter is maintained via RDMA FAA by active readers, preventing reclamation while a transaction still holds a reference to the record.

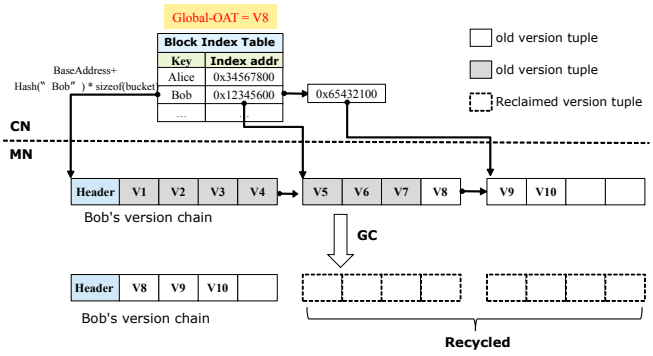


Figure 7: Garbage collection process illustrating the background compaction path. A version chain is pruned based on Global-OAT visibility, consolidating live versions (V8–V10) into a new block while reclaiming obsolete deltas.

MiTDM utilizes this strict safety margin via two independent GC pathways (Figure 7):

- **Fast Path (In-place Reuse):** Local transactions preferentially reuse slots whose versions are older than Global-OAT, reducing fragmentation within existing blocks.
- **Slow Path (Background Compaction):** The GC thread acquires the per-record lock, relocates live versions into a compact Overflow Block, relinks the chain, and returns retired physical blocks to the compute node's global bitmap-managed memory pool.

3.6 Transaction Protocol

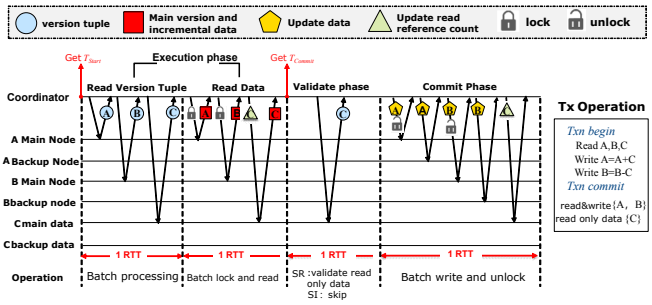


Figure 8: Transaction lifecycle in MiTDM walkthrough: (1) Execute phase fetches variables using T_{start} to identify the visible T_c, aborting early under serializable isolation if a newer committed version is observed, and acquires write locks for the write set via RDMA CAS; (2) Validate phase rechecks that read-only data remain unchanged before commit under serializable isolation; and (3) Commit phase publishes metadata with T_{new} as the atomic visibility point.

MiTDM employs an optimistic transaction protocol proceeding through three strictly structured phases, mapping the lifecycle timestamps (T_{start}, T_c, and T_{new}) identically modeled in Figure 8:

Execute ($T_{start} \rightarrow T_c$): Initiated with a globally allocated T_{start}, the transaction searches the tuple using the local BIT, fetching raw block data via RDMA `READ`. It locally selects the visible version where its commit timestamp $T_c \leq T_{start}$. Under serializable isolation, if the version scan observes any committed version with timestamp greater than T_{start} before selecting the target version, the coordinator aborts immediately instead of carrying a stale read into validation; under snapshot isolation, such newer versions are ignored for read visibility. For records retained in the read set, the coordinator batches an RDMA FAA on the tuple header to increment `read_cnt`, preventing reclamation while the transaction remains active. For items in the write set, the coordinator acquires the per-record lock via RDMA CAS and reconstructs the target version payload locally before buffering updates in a private workspace.

Validate (T_c verification): This stage enforces the selected isolation level for transactions that survive Execute. Under serializable isolation, the coordinator revalidates that read-only data observed at T_{start} have not changed before commit; if any validation fails, the transaction aborts and releases its locks and read references. Under snapshot isolation, this read-only validation can be skipped, while the write-set locks acquired during Execute still prevent concurrent write-write conflicts.

Commit (Assigning T_{new}): After obtaining a commit timestamp T_{new} from the timestamp service, the transaction applies updates using a strictly ordered sequence that leverages RDMA NIC ordering guarantees [14, 26]: (1) write reverse-delta payloads first,

Table 1: Experimental environment configuration.

Component	Configuration
CPU model	2x Intel Xeon Gold 6230R @ 2.10 GHz
Cores per machine	104 hardware threads
DRAM	256 GB DDR4
RDMA switch	MSB7890-ES2F 100 Gbps InfiniBand
RDMA NIC	Mellanox ConnectX-5
OS	Ubuntu 20.04.5 LTS
Kernel	Linux 5.4.0-135-generic (x86_64)

which serve as undo logs for transaction abort handling; (2) write the new materialized main data; (3) update the Version Cell metadata, including the new commit timestamp and the `VerStart/End` checksum counters; and (4) release the per-record lock via RDMA `WRITE`. The metadata update in step three is the atomic publish point. Readers accept a version only after strict timestamp and checksum integrity validation, naturally ignoring partial writes.

To close the GC-safety lifecycle, the transaction decrements the tuple-header `read_cnt` via RDMA `FAA` when it commits or aborts, signaling to the background GC thread that the record is no longer pinned by the reader.

4 EVALUATION

We evaluate MiTDM on a disaggregated memory (DM) cluster, covering end-to-end OLTP performance, design-specific micro-benchmarks, and parameter sensitivity.

4.1 Experimental Setup

4.1.1 Hardware and Software Environment. Our testbed consists of five servers connected by 100 Gbps InfiniBand (Table 1). Although hardware-constrained, this scale is consistent with prior RDMA disaggregated systems such as Motor, FORD, and FaRM.

To emulate a disaggregated memory architecture, we configure three machines as *compute nodes* and the other two as *memory pool nodes*. Unless otherwise stated, all experiments use this default deployment of three compute nodes and two memory pool nodes, while Section 4.3.2 and Section 4.4.2 also evaluate scalability by varying the compute nodes from one to three. Each compute node uses all CPU cores with local DRAM capped at 8 GB. The two memory nodes expose their full 256 GB DRAM as remote memory. Data tables are stored entirely in this remote memory pool, achieving fault tolerance via a two-replica configuration.

4.1.2 Compared Systems and MiTDM Configuration. We compare MiTDM against two baselines implemented in the same RDMA-based framework [14]:

- **FaRMV2-DM.** We port FaRM-V2's chain-based MVCC design to DM by reorganizing version tuples into an N2O-style linked list. This baseline highlights the overhead of version-chain traversal across the network.
- **Motor-4V / Motor-9V.** Motor introduces contiguous version tuples (CVTs), fixed-size arrays that co-locate multiple versions per key. We implement two configurations with different capacities: 4 and 9 versions. This exposes the trade-off between false conflicts and inflated transfer size.

MiTDM uses HBVC, asynchronous Block Index Table synchronization, and state-aware hybrid GC. Unless stated otherwise, the default block capacity is four version slots and the GC trigger threshold is 8 MB.

4.2 Micro-Benchmark

We evaluate TPC-C, TATP, and SmallBank to cover long transactions, light contention, and write-heavy skew.

4.2.1 TPC-C. Figure 9a shows throughput scaling, and Figure 10 reports P50/P99 latencies. At 128 clients, MiTDM outperforms FaRMV2-DM by 80.1% and reduces P50/P99 latency by 30.6%/35.3%. TPC-C amplifies chain-traversal cost because each write touches about 13 records across 9 tables, whereas MiTDM fetches several versions per batched RDMA read.

Against Motor-4V, MiTDM improves throughput by 42.7% and lowers P99 latency by 27.4%, while keeping similar P50 latency. Motor-4V collapses beyond 64 clients because its fixed four-slot array overwrites still-visible versions. Motor-9V reduces this problem with nine slots, but its larger arrays still leave MiTDM ahead by up to 31.6% in throughput and by 5.1%/15.7% in P50/P99 latency. MiTDM expands only hot records, while Motor inflates every read.

This difference is especially visible in `STOCK_LEVEL`, where a long read set magnifies unnecessary version transfer. Motor-9V mitigates false conflicts by provisioning larger per-key arrays, but it pays that space cost on every access, including cold keys and short histories. MiTDM instead keeps the common path compact and allocates overflow capacity only when a record actually becomes hot.

4.2.2 TATP. At 128 clients, MiTDM achieves 24.1% higher throughput than FaRMV2-DM and reduces P50/P99 latency by 16.2%/30.5%. The gap is smaller than in TPC-C because TATP is read-dominated (20% writes), touches only 2–3 records per transaction, and rarely creates long chains or array overflows. MiTDM also remains slightly better than Motor-4V and Motor-9V, but the margin is modest because false conflicts are infrequent and larger arrays are less harmful in this workload.

This result is useful because it shows MiTDM is not merely tuned for pathological contention. Even when the workload already favors compact fixed arrays, HBVC preserves competitive latency while avoiding the rigidity that hurts array-based layouts once contention increases.

4.2.3 SmallBank. SmallBank is write-intensive and skewed. At 128 clients, MiTDM improves throughput over FaRMV2-DM by 36.8% and lowers P50/P99 latency by 34.1%/50.3%, because it avoids repeated remote pointer chasing on rapidly growing chains.

MiTDM also outperforms Motor-4V by 30.2% in throughput and 41.8% in P99 latency, despite slightly higher P50 latency in some highly skewed cases. Motor-4V runs out of version capacity on hot keys, while Motor-9V pays for nine-slot arrays on every read. As a result, MiTDM still leads Motor-9V by 22.1% in throughput and by 11.3%/20.9% in P50/P99 latency at 128 clients.

The slightly higher P50 latency in a few skewed cases comes from deeper overflow chains on extremely hot accounts, where MiTDM may fetch more than one block. However, avoiding aborts and retries matters more for overall service quality: once contention rises, lower tail latency and higher sustained throughput dominate the small median penalty.

4.3 Macro-Benchmarks: Evaluating Core Designs

We now isolate the effects of HBVC, asynchronous index synchronization, and hybrid GC using YCSB-based KVS micro-benchmarks.

4.3.1 Effect of Block-based Version Chains. We first quantify how HBVC affects lookup latency and false conflicts.

Target version lookup latency. We compare the N2O chain from FaRMV2-DM, Motor arrays with 4 and 9 slots, and MiTDM's HBVC on YCSB KVS with Zipfian access ($\alpha = 1.0$), four read/write ratios, and 128 clients. We measure target-version lookup latency for both single-key transactions and multi-key transactions that read 10 records.

In the single-key setting (Figure 11a), MiTDM lowers lookup latency by 1.79×–3.14× over the N2O chain by fetching several candidate versions in one RDMA operation. Motor-4V can still be slightly faster when histories are short because one compact array is ideal for that case. In the multi-key setting (Figure 11b), MiTDM becomes the best design: it is 1.62×–3.87× faster than N2O and 1.17×–1.22× faster than the nine-version array, because it avoids both per-hop pointer chasing and globally inflated arrays.

This crossover between single-key and multi-key workloads highlights the main design point of HBVC. Pure chains waste RTTs, and large fixed arrays waste bandwidth. HBVC avoids both extremes by batching the common case while expanding only where history depth really requires it.

false conflicts in long-running read-only transactions. To expose false conflicts, we construct long read-only transactions that scan 200 keys while concurrent read/write transactions follow a Zipfian distribution. Figure 12a shows that Motor-4V reaches a 68.4% abort rate at a 2:8 read/write ratio, and Motor-9V still reaches 30.4%. In contrast, both FaRMV2-DM and MiTDM maintain a 0% abort rate across all ratios because their version structures grow elastically and preserve old versions until GC can safely reclaim them.

This behavior matches the intended semantics of MVCC: long read-only transactions should remain non-blocking even under concurrent updates. MiTDM preserves that property without inheriting the latency cost of fully pointer-based chains.

4.3.2 Asynchronous Block Index Table Synchronization. MiTDM maintains per-key Block Index Table entries at compute nodes and propagates updates asynchronously via notification queues. We compare this design with a *synchronous* scheme that waits for all compute nodes to install a new index entry before commit completes.

On YCSB KVS configured with three compute nodes (3CN) and two memory nodes, using 128 clients and Zipfian access ($\alpha = 1.0$), the asynchronous scheme delivers 1.14×–1.83× higher throughput (Figure 12b). The advantage widens as writes become more frequent: synchronous propagation extends critical sections and lock hold times on hot keys, whereas the asynchronous design removes index propagation from the commit path.

The gap is small under read-heavy ratios because new block allocation is rare when reuse and GC absorb most updates. Once writes become frequent, however, synchronous propagation turns every structural update into an avoidable distributed stall, undermining the benefit of a passive-memory design.

Another practical benefit is isolation of foreground latency from control-plane work. With asynchronous propagation, writers only append notifications and proceed, while background workers absorb queue processing and local table repair. This keeps the fast path simple and prevents remote metadata maintenance from dominating commit latency.

4.3.3 State-aware Hybrid Garbage Collection. MiTDM employs state-aware hybrid GC, combining in-place reuse with background compaction under a Global-OAT watermark. We evaluate its impact under sustained write pressure.

We drive the KVS with a 128-thread pure-write workload and compare four configurations: no GC, background GC only, in-place reuse only, and MiTDM's hybrid GC.

Figure 13 shows that without GC, throughput steadily declines as chains grow. Background GC alone recovers performance but introduces periodic dips of about 17.1% during compaction. In-place reuse slows the decline, but it cannot shorten existing chains or reduce transferred metadata. Hybrid GC combines both benefits: reuse postpones compaction, and targeted background reclamation restores compact chains when the Block Index Table exceeds the threshold, keeping throughput close to peak for most of the run.

In other words, reuse controls allocation pressure, while background compaction controls lookup cost. Neither mechanism alone is sufficient over long executions; the combined design is what keeps MiTDM from degrading as version history accumulates.

This result matters for deployment because long-running OLTP services rarely enjoy clean restart points. A design that performs well only in short runs can overstate its practical performance if it degrades as historical state accumulates. Hybrid GC keeps MiTDM stable over time rather than merely fast at the beginning of the run.

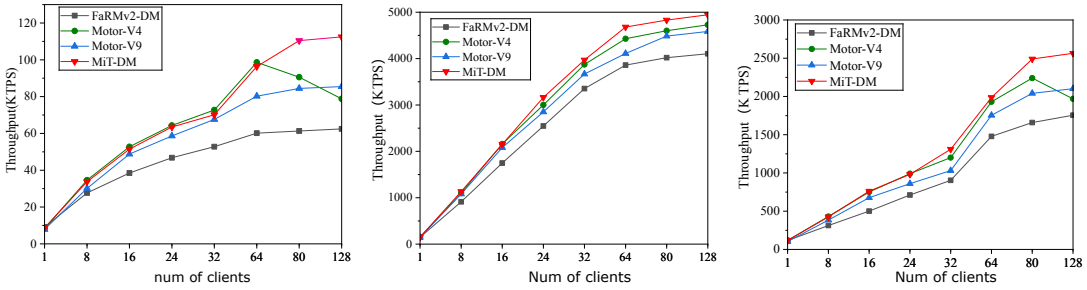
4.4 Sensitivity Analysis

We conclude with a sensitivity study of block size, compute-node count, and GC trigger threshold.

4.4.1 Block Size. Varying block size from 1 to 12 (Figure 14) reveals the expected trade-off. A size of one degenerates HBVC into a pure chain, while excessively large blocks inflate each RDMA read. The penalty is strongest in multi-key workloads, where TPC-C throughput drops by up to 35.8% beyond the sweet spot; TATP and SmallBank see smaller drops of 10.5% and 17.4%. We therefore use a default block size of 4, which stays near peak across all workloads.

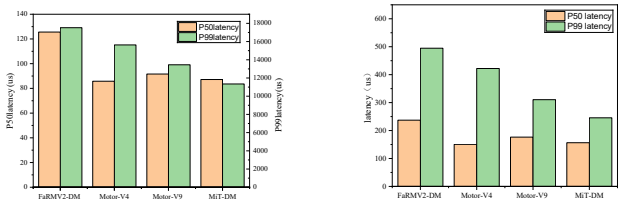
This result also explains why simply scaling Motor-style arrays is the wrong fix for false conflicts: larger containers help only up to the point where transfer overhead starts dominating, and that point arrives quickly in multi-key transactions.

4.4.2 Number of Compute Nodes. We scale compute nodes from one to three (while keeping the two memory nodes fixed) with



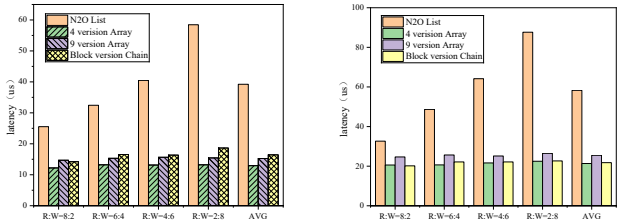
(a) TPC-C throughput. (b) TATP throughput. (c) SmallBank throughput.

Figure 9: Throughput vs. number of clients for TPC-C, TATP, and SmallBank (one-column layout).



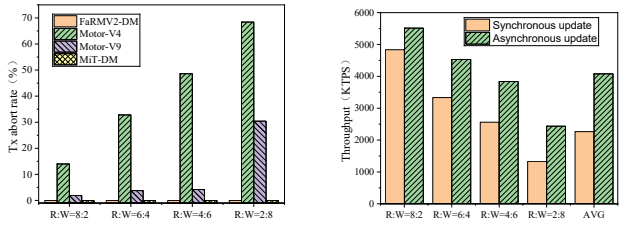
(a) TPC-C P50 and P99 latencies at 128 clients. (b) SmallBank P50 and P99 latencies at 128 clients.

Figure 10: Latency comparison for TPC-C and SmallBank at 128 clients.



(a) Single-key transactions. (b) Multi-key transactions.

Figure 11: Average target-version lookup latency under different read/write ratios in KVS workloads.



(a) Abort rate of long read-only transactions (200 keys) under different read/write ratios. (b) Throughput under different index synchronization schemes (128 clients).

Figure 12: Performance of core designs under contention.

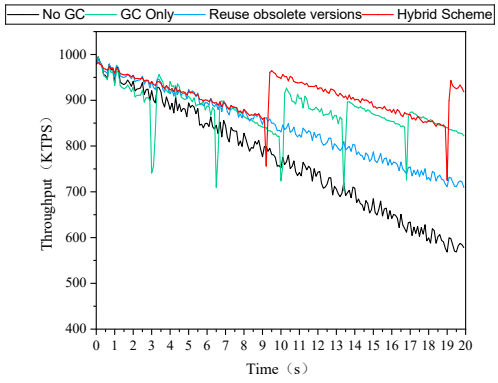


Figure 13: Throughput over time under different GC strategies (128-thread pure write workload).

128 evenly distributed clients. Figure 15a shows that high write ratios slightly increase read latency, by at most 8.7% at three nodes, because some reads encounter stale index entries and fall back to chain traversal. Even so, the increase remains small, indicating that asynchronous synchronization scales well within our hardware limit.

The key point is that staleness affects only the fast path, not correctness. When an entry is outdated, MiTDM falls back to chain traversal and repairs the local view, so synchronization delay manifests as a modest latency tax rather than a semantic hazard.

4.4.3 GC Trigger Threshold. We vary the GC trigger threshold from 4 KB to 64 MB under a write-heavy KVS workload (Figure 15b). Small thresholds trigger frequent GC and visible throughput jitter, while very large thresholds let obsolete index entries accumulate and lengthen traversals. The best range is 4–8 MB, so MiTDM uses 8 MB by default.

This threshold study reflects the same balancing act as the rest of MiTDM: overly aggressive cleanup interferes with foreground work, while lazy cleanup quietly bloats the metadata path. The selected default keeps both effects in check.

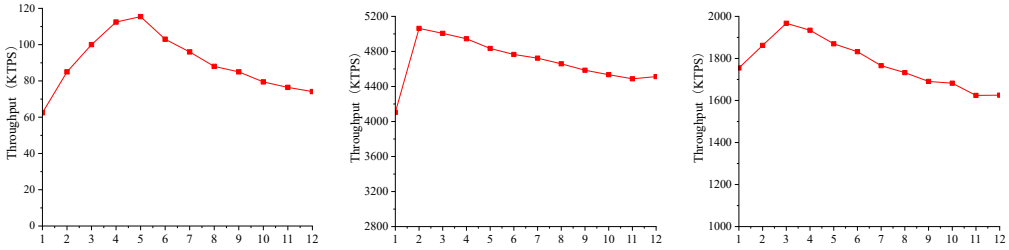
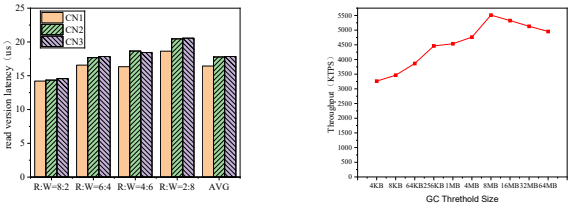


Figure 14: Throughput vs. block size under different workloads.



(a) Average version read latency vs. number of compute nodes. **(b) Throughput vs. GC trigger threshold.**

Figure 15: Sensitivity analysis

5 RELATED WORK

5.1 Disaggregated Memory Transaction Systems

Our work builds upon distributed transaction systems for DM. Early systems like FaRM [5] demonstrated low-latency RDMA-based transactions using single-version CC, while subsequent works such as NAM-DB [37], Xenic [27], FORD [39], and Scythe [21] explored a broader set of storage and communication optimizations. However, single-version systems inevitably sacrifice concurrency under long or overlapping transactions [36], which motivates the use of MVCC in disaggregated settings [28, 31, 38].

Chain-based MVCC systems. FaRM-V2 [28], Aurogon [12], NAM-DB [37], and RCC [31] organize versions in linked lists. This design naturally provides unbounded history capacity, but traversing chains requires dependent RDMA reads and thus incurs network-bound pointer chasing. The problem worsens when records are hot or transactions read many keys, because version lookup latency scales with the number of remote hops.

Array-based MVCC systems. Motor [38] stores versions in fixed-size contiguous arrays, enabling one-RTT batched fetches and excellent spatial locality in the common case. However, the same rigidity in per-key capacity creates *false conflicts* when long-running transactions need versions that have already been overwritten. Enlarging arrays can alleviate this issue, but it also inflates transfer size for every read, including keys that never need the extra capacity.

MiTDM resolves this trade-off via HBVC, which combines batched block access with dynamic capacity expansion and thus avoids both per-version pointer chasing and globally oversized version containers.

5.2 Indexes and Garbage Collection

Index Structures. DM index research spans hash-based [3, 23, 24, 42], tree-based [1, 19, 32, 35], and hybrid designs [17, 22, 33]. These works primarily optimize primary-key lookup latency and update scalability. They are orthogonal to MiTDM's version-management problem, and MiTDM can readily use any of these structures for primary-key access.

Garbage Collection. Distributed MVCC GC typically requires cluster-wide coordination [4, 20], and memory-side compaction is impractical in DM due to compute asymmetry [9, 16]. MiTDM instead exploits compute-side transaction state to reclaim versions without memory-node coordination, allowing reclamation to remain low-interference even when memory pools are passive and remote CPU involvement is unavailable.

6 CONCLUSION

We presented MiTDM, an MVCC system for disaggregated memory that resolves the long-standing trade-off between access latency and storage elasticity. By combining the Hierarchical Block-based Version Chain (HBVC), asynchronous index synchronization, and state-aware hybrid GC, MiTDM enables low-latency batched RDMA reads while still supporting dynamic version expansion without false conflicts. Across our workloads, MiTDM improves throughput by up to 80.1% over chain-based designs and 42.7% over array-based designs, while eliminating aborts in long-running read-only queries caused by rigid version storage. These results suggest that DM-oriented MVCC should not choose between chains and arrays as two extremes; a hierarchical design can preserve the strengths of both while avoiding their dominant pathologies.

REFERENCES

[1] Hao An, Feifei Wang, Deng Feng, Xinyu Zou, Zhaolei Liu, and Jie Zhang. 2023. Marlin: A Concurrent and Write-Optimized B+-tree Index on Disaggregated Memory. In *52nd International Conference on Parallel Processing (ICPP)*. ACM, 695–704.

[2] Wei Cao, Yiying Zhang, Xin Yang, Feifei Li, Sitong Wang, Qing Hu, et al. 2021. PolarDB Serverless: A Cloud Native Database for Disaggregated Data Centers. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*. ACM, 2477–2489.

[3] Yuanwei Chen, Xingda Wei, Jiaji Shi, Rong Chen, and Haibo Chen. 2016. Fast and general distributed transactions using RDMA and HTM. In *Eleventh European Conference on Computer Systems (EuroSys)*. ACM, 1–17.

[4] Bailu Ding, Lucja Kot, Alan Demers, and Johannes Gehrke. 2015. Centiman: Elastic, High Performance Optimistic Concurrency Control by Watermarking. In *Proceedings of the Sixth ACM Symposium on Cloud Computing (SoCC)*. ACM, 262–275.

[5] Aleksandar Dragojević, Dushyanth Narayanan, Miguel Castro, and Antony Rowstron. 2014. FaRM: Fast Remote Memory. In *11th USENIX Symposium on*

Networked Systems Design and Implementation (NSDI 14). USENIX Association, 401–414.

[6] Franz Färber, Sang Kyun Cha, Jürgen Primsch, Christof Bornhövd, Stefan Sigg, and Wolfgang Lehner. 2012. SAP HANA database: data management for modern business applications. *ACM Sigmod Record* 40, 4 (2012), 45–51.

[7] Yaniv Fridman, Shreyas M. Desai, Nitya Singh, Tom Willhalm, and Gidon Oren. 2023. CXL Memory as Persistent Memory for Disaggregated HPC: A Practical Approach. In *SC-W 2023: Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*. IEEE Computer Society, 983–994.

[8] Daniel Gouk, Sungmin Lee, Myeongjae Kwon, and Myoungsoo Jung. 2022. DirectCXL: Direct Access, High-Performance Memory Disaggregation with DirectCXL. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. USENIX Association, 287–294.

[9] Zhaoguo Guo, Yizhou Shan, Xiaowei Luo, Yutong Huang, and Yiying Zhang. 2022. Clio: a hardware-software co-designed disaggregated memory system. In *27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 417–433.

[10] Zhaoguo Guo, Kang Wu, Cong Yan, and Xiangyao Yu. 2021. Releasing Locks As Early As You Can: Reducing Contention of Hotspots by Violating Two-Phase Locking. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*. ACM, 658–670.

[11] Dongxu Huang, Qi Liu, Qiu Cui, Zhuhe Fang, Xiaoyu Ma, Fei Xu, Li Shen, Liu Tang, Yuxing Zhou, Menglong Huang, et al. 2020. TiDB: a Raft-based HTAP database. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3072–3084.

[12] Tianyang Jiang, Guangyan Zhang, Zhiyue Li, and Weimin Zheng. 2022. Aurogon: Taming Aborts in All Phases for Distributed In-Memory Transactions. In *20th USENIX Conference on File and Storage Technologies (FAST 22)*. USENIX Association, 217–232.

[13] Anuj Kalia, Michael Kaminsky, and David G. Andersen. 2014. Using RDMA efficiently for key-value services. In *Proceedings of the 2014 ACM SIGCOMM Conference*. ACM, 295–306.

[14] Anuj Kalia, Michael Kaminsky, and David G. Andersen. 2016. Design guidelines for high performance RDMA systems. In *2016 USENIX Annual Technical Conference (USENIX ATC 16)*. USENIX Association, 437–450.

[15] Kyriakos Katrinis, Dionisios Syrivelis, Dimitrios Pnevmatikatos, Georgios Zervas, Dimitris Theodoropoulos, Ilias Koutsopoulos, et al. 2016. Rack-scale disaggregated cloud data centers: the dReDBox project vision. In *2016 Conference on Design, Automation & Test in Europe (DATE)*. IEEE, 690–695.

[16] Sungmin Lee, Yuhan Yu, Yifan Tang, Arvind Panda, Sylvia Ratnasamy, and Scott Shenker. 2021. Mind: In-network memory management for disaggregated data centers. In *ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*. ACM, 488–504.

[17] Peng Li, Yuxin Hua, Pengfei Zuo, Zhaoguo Chen, and Jiacheng Sheng. 2023. ROLEX: A scalable RDMA-oriented learned key-value store for disaggregated memory systems. In *21st USENIX Conference on File and Storage Technologies (FAST 23)*. USENIX Association, 99–114.

[18] Kihwan Lim, Jichuan Chang, Trevor Mudge, Parthasarathy Ranganathan, Steven K. Reinhardt, and Thomas F. Wenisch. 2009. Disaggregated memory for expansion and sharing in blade servers. In *Proceedings of the 36th Annual International Symposium on Computer Architecture, ISCA 2009*. ACM, 267–278.

[19] Boyu Lu, Keren Huang, Chun Jason M. Liang, Tianzheng Wang, and Eugene Lo. 2024. DEX: Scalable Range Indexing on Disaggregated Memory. *Proc. VLDB Endow.* 17, 10 (2024), 2603–2616.

[20] Haosen Lu, Christopher Hodsdon, Khiem Ngo, Shuai Mu, and Wyatt Lloyd. 2016. The SNOW Theorem and Latency-Optimal Read-Only Transactions. *Operating Systems Design and Implementation (OSDI)* (2016), 135–150.

[21] Keyong Lu, Shengan Zhao, Haotian Shan, Qiao Wei, Gang Li, Jian Wan, et al. 2024. Scythe: A Low-latency RDMA-enabled Distributed Transaction System for Disaggregated Memory. *ACM Trans. Archit. Code Optim.* 21, 3 (2024), 1–26.

[22] Xinyu Luo, Pengfei Zuo, Jiacheng Shen, Jiale Gu, Xusheng Wang, Michael R. Lyu, et al. 2023. SMART: A High-Performance Adaptive Radix Tree for Disaggregated Memory. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. USENIX Association, 553–571.

[23] Xingda Min, Keyong Lu, Peilei Liu, Jian Wan, Chang Xie, Di Wang, et al. 2024. SepHash: A Write-Optimized Hash Index On Disaggregated Memory via Separate Segment Structure. *Proc. VLDB Endow.* 17, 5 (2024), 1091–1104.

[24] Christopher Mitchell, Yifeng Geng, and Jinyang Li. 2013. Using {One-Sided}{RDMA} Reads to Build a Fast,{CPU-Efficient}{Key-Value} Store. In *2013 USENIX Annual Technical Conference (USENIX ATC 13)*. 103–114.

[25] Christopher Mitchell, Yifeng Geng, and Jinyang Li. 2015. Using One-Sided RDMA Reads to Build a Fast, CPU-Efficient Key-Value Store. In *2015 USENIX Annual Technical Conference (USENIX ATC 15)*. 103–114.

[26] Waleed Reda, Marco Canini, Dejan Kostic, and Simon Peter. 2022. RDMA is Turing Complete, We Just Did Not Know It Yet!. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. USENIX Association, 71–85.

[27] Henry N. Schuh, Weiyuan Liang, Minjae Liu, Jacob Nelson, and Arvind Krishnamurthy. 2021. Xenic: SmartNIC-Accelerated Distributed Transactions. In *ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*. ACM, 740–755.

[28] Aleksey Shamis, Matthew Renzelmann, Stanko Novakovic, Georgios Chatzopoulos, Aleksandar Dragojević, Dushyanth Narayanan, et al. 2019. Fast General Distributed Transactions with Opacity. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*. ACM, 433–448.

[29] Yizhou Shan, Yutong Huang, Yilun Chen, and Yiying Zhang. 2018. LegoOS: A Disseminated, Distributed OS for Hardware Resource Disaggregation. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, 69–87.

[30] Stephen Tu, Wenting Zheng, Evan Kohler, Barbara Liskov, and Samuel Madden. 2013. Speedy transactions in multicore in-memory databases. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles (SOSP)*. ACM, 18–32.

[31] Cheng Wang and Xiaojian Qian. 2023. RDMA-Enabled Concurrency Control Protocols for Transactions in the Cloud Era. *IEEE Transactions on Cloud Computing* 11, 1 (2023), 798–810.

[32] Qing Wang, Yifei Lu, and Jiwu Shu. 2022. Sherman: A Write-Optimized Distributed B+Tree Index on Disaggregated Memory. In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD)*. ACM, 1033–1048.

[33] Ruize Wang, Chengying Gao, Jian-Jia Wang, Prasad Kadam, M. Tamer Özsu, and Walid G. Aref. 2024. Optimizing LSM-based indexes for disaggregated memory. *The VLDB Journal* 33, 6 (2024), 1813–1836.

[34] Xingda Wei, Rong Chen, Haibo Chen, Zhaoguo Wang, Zhenhan Gong, and Binyu Zang. 2021. Unifying Timestamp with Transaction Ordering for MVCC with Decentralized Scalar Timestamp. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*. USENIX Association, 357–372.

[35] Xingda Wei, Zhiyuan Dong, Rong Chen, and Haibo Chen. 2021. Characterizing and Optimizing Remote Persistent Memory with RDMA and NVM. *ACM Trans. Storage* 17, 3 (2021), 1–26.

[36] Ebrahim Zamanian, Joy Arulraj, Chunbin Binnig, Tim Kraska, and Tim Harris. 2020. Chiller: Contention-centric Transaction Execution and Data Partitioning for Modern Networks. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. ACM, 511–526.

[37] Ebrahim Zamanian, Carsten Binnig, Tim Harris, and Tim Kraska. 2017. The end of a myth: distributed transactions can scale. *Proc. VLDB Endow.* 10, 6 (2017), 685–696.

[38] Mengzhe Zhang, Yuxin Hua, and Ziyang Yang. 2024. Motor: Enabling Multi-Versioning for Distributed Transactions on Disaggregated Memory. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 801–819.

[39] Mengzhe Zhang, Yuxin Hua, Pengfei Zuo, and Lurong Liu. 2022. FORD: Fast One-sided RDMA-based Distributed Transactions for Disaggregated Persistent Memory. In *20th USENIX Conference on File and Storage Technologies (FAST 22)*. USENIX Association, 51–68.

[40] Yiying Zhang, Chen Ruan, Cheng Li, Xin Yang, Wei Cao, Feifei Li, et al. 2021. Towards cost-effective and elastic cloud database deployment via memory disaggregation. *Proc. VLDB Endow.* 14, 10 (2021), 1900–1912.

[41] Yifan Zhong, Mingkai Zhou, Ziyue Shen, and Jiwu Shu. 2024. UniMem: Redesigning Disaggregated Memory within a Unified Local-Remote Memory Hierarchy. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. USENIX Association, 463–477.

[42] Pengfei Zuo, Jiacheng Sun, Letian Yang, Sijie Zhang, and Yuxin Hua. 2021. One-sided RDMA-Conscious Extendible Hashing for Disaggregated Memory. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, 15–29.